

PolyCalc 架构概览

1. 设计目标

传统数学定理通常描述一个确定结论，例如“极限等于某个实数”

但符号计算的中间结果可能具有不同性质和精度：

- 精确的普通数值
- 正无穷、负无穷或符号未知的无穷
- 已知发散，但不知道更具体的发散形式
- 当前信息不足，只能得到未知结果

如果系统只允许返回普通数值，那么每一步计算都必须提前解决非零性、符号、定义域和收敛性等条件，复杂表达式将很难组合

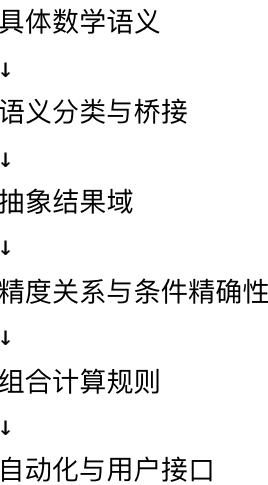
本系统采用多精度结果设计：

中间计算可以返回保守但正确的抽象结果，只有在最终需要精确结论时，才恢复必要条件和普通等式

这样既能让计算持续进行，又不会用没有数学意义的任意值掩盖信息不足

2. 总体架构

系统可以分为六层：



2.1 具体数学语义

底层使用传统数学命题定义收敛、无穷极限、连续和可导等性质

例如，有限极限、正无穷极限和负无穷极限首先都是独立而严格的数学命题，它们构成表达式计算的语义基础

2.2 语义分类与桥接

系统把具体数学事实分类为统一的表达式结果，并在传统命题和表达式结论之间建立双向联系

极限分类器的核心形式如下：

```
def GenericExpr (C P N U : Prop)
  [laws : GenericExprLaws C P N U] : LimitValue :=
  if h : C then the (laws.evC h)
  else if P then pos_infty
  else if N then neg_infty
  else if U then infty
  else diverg
```

其中 C 、 P 、 N 、 U 分别表示有限收敛、正无穷、负无穷和无符号无穷

分类顺序体现了结果精度，若能够确定有限值，就不会退化为无穷、发散或未知

桥接机制负责完成：

传统数学命题 $\rightleftharpoons$ 表达式计算结论

因此，表达式系统不是对传统定义的替代，而是建立在传统定义之上的计算接口

2.3 抽象结果域

抽象结果域统一承载不同种类和不同精度的结果

极限系统使用：

```
inductive LimitValue
| finite (a : ℝ)
| unknownLimit
| posInfty
| negInfty
| unsignedInfty
| divergence
```

这些构造子分别表示精确有限值、未知、正无穷、负无穷、无符号无穷和发散

每个结果都代表一个有意义的数学观察，而不是错误码

例如，“正无穷”比“无符号无穷”更精确，“无符号无穷”又比“发散”更精确，“未知”则表示当前不能给出更强结论

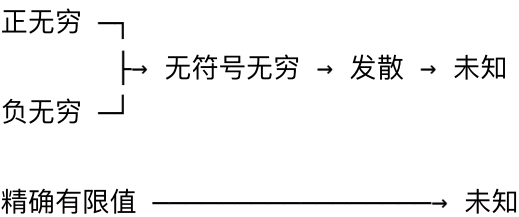
2.4 精度关系与条件精确性

抽象结果之间存在从精确到粗略的定向关系

极限系统先声明领域特有的一步退化：

```
inductive LimitFallbackCore : LimitValue → LimitValue → Prop
| infty_divergence :
  LimitFallbackCore .unsignedInfty .divergence
| pos_infty_infty :
  LimitFallbackCore .posInfty .unsignedInfty
| neg_infty_infty :
  LimitFallbackCore .negInfty .unsignedInfty
```

结合任意结果都可以退化到未知值这一通用规则，可得到如下精度结构：



箭头表示安全的信息遗忘，而不是普通等式，因此通常不可逆

系统还区分精确结果，即足够精确且具有刚性的结果

在极限系统中只有有限值被认为是精确的：

```
def LimitValue.isProper : LimitValue → Prop
| .finite _ => True
| _ => False
```

如果一个抽象结果可以退化为精确结果，那么两者实际上必须完全相同，由此可以恢复普通等式

2.5 组合计算规则

这一层描述复合表达式如何由子表达式计算，例如极限的和、积、商以及导数的运算法则

抽象运算直接定义在结果域上，例如有限值相加仍然是有限值：

```
instance : Add LimitValue where
add A B := match A, B with
| the a, the b => the (a + b)
| the _, pos_infty => pos_infty
| pos_infty, the _ => pos_infty
| _, _ => unknown
```

实际运算表还会保留更多可确定的分支，这里只展示基本思想

当输入信息足够时，运算返回精确结果，当遇到无法安全判断的不定形式时，运算返回较粗结果或未知

组合规则通常给出精度关系，而不是普通等式，因为分别计算子表达式可能丢失它们之间的相关信息

例如，两个看似方向相反的无穷量相加时，抽象运算可能只能返回未知，但原表达式可能因精确消去而具有有限结果

因此通常只能保证：

「直接分析整个表达式」至少与「分别分析子表达式后再组合」一样精确

2.6 自动化与用户接口

最上层负责把上述结构包装成接近普通数学计算的证明体验

自动化主要完成：

- 在表达式中应用合适的计算规则
- 将子表达式的精度关系传播到外层运算

- 从复合结果的精确性反推出子结果所需条件
- 在结果精确时恢复普通等式并执行重写

3. 三种相等关系

系统联合使用普通等式 Eq、多态相等 PE 和精确时相等 EIP

它们的强度和用途不同：

关系	含义	典型用途
$A = B$	两个值完全相同	普通重写、代入和精确结论
$A =. B$	A 可以安全降低精度为 B	多精度表达式计算
$A =? B$	当 B 精确时有 $A =. B$	带延迟 side condition 的计算规则

3.1 普通等式 =

普通等式是 Lean 中的严格相等关系

它是对称的，可以在任意上下文中直接重写，也是系统最终希望恢复的精确结论形式

3.2 多态相等 PolyEq

多态相等使用记号：

$A =. B$

它表示 A 至少和 B 一样精确，或者说 A 可以通过零步或多步合法信息退化得到 B

它具有自反性和传递性，但通常不具有对称性

例如：

- 正无穷 $=.$ 无符号无穷
- 无符号无穷 $=.$ 发散
- 正无穷 $=.$ 发散

反方向一般不成立，因为已经遗忘的信息不能凭空恢复

当 B 精确时, $A =. B$ 等同于普通等式 $A = B$

3.3 精确时相等 `EqualIfProper`

精确时相等使用记号:

$$A =? B$$

其含义是:

如果 B 精确, 那么 $A =. B$

它适合描述只在符号右式最终足够精确时才成立的公式

例如, 导数的乘法、除法和链式法则可能需要右侧的子导数有限、分母非零或相关函数连续, 因此可以先用 $=?$ 表述规则, 把这些条件延迟到最终需要精确结果时再恢复

必须注意, $A =? B$ 不等于无条件的 $A =. B$

若 B 不精确, 条件命题可能空真, 当且仅当取得 B 精确的证明后, 此结果自动退化为多态相等, 并可进一步恢复普通等式

4. 联合传递性

三种关系能够在同一条链式计算中组合

4.1 $=$ 与 $=.$ 的组合

普通等式不会改变精度关系, 因此:

$$A = B, B =. C \Rightarrow A =. C$$

$$A =. B, B = C \Rightarrow A =. C$$

$$A =. B, B =. C \Rightarrow A =. C$$

这允许计算链在精确代数变形和抽象精度步骤之间自由切换

概念上可以写成:

```

calc
  A = B := exactStep1
  _ =. C := abstractStep
  _ = D := exactStep2

```

整条链的最终关系是 $A =. D$

4.2 =? 与其他关系的组合

精确时相等也可以与三种关系联合传递：

```

A = B, B =? C   => A =? C
A =? B, B = C   => A =? C
A =? B, B =? C => A =? C
A =. B, B =? C => A =? C
A =? B, B =. C => A =? C

```

这些规则的共同思路是先假设最终右端 C 精确

若有 $B =. C$ ，则 C 的精确性会迫使 $B = C$

这样就可以把链中间的精度关系收缩为普通等式，再把前一段结论运输到最终右端

因此， $=?$ 可以作为整条条件计算链的外层关系，同时允许链内混合普通等式和多态相等

4.3 三种关系的使用原则

- 能证明完全相同时使用 $=$
- 计算可能安全丢失信息时使用 $=.$
- 规则只当「可靠性建立在最终结果精确上」时使用 $=?$
- 一旦右端精确，就把 $=.$ 或 $=?$ 恢复成普通等式再进行任意重写

5. 两种信息传播方向

系统同时使用两种方向相反的信息传播

5.1 正向精度传播

如果输入可以安全降低精度，抽象运算的输出也应能够相应降低精度：

A 更精确于 A' , B 更精确于 B'
↓
 $op(A, B)$ 更精确于 $op(A', B')$

这使计算规则能够在复杂表达式内部组合和局部应用

5.2 逆向条件恢复

如果一个复合结果已知是精确的，系统可以反推出相关子结果必须满足的必要条件：

复合结果精确
↓
子结果有限、分母非零、函数连续等

正向传播用于继续抽象计算，逆向恢复则用于在最终阶段重新获得被延迟的 side condition

6. 架构优势

- **总函数化计算**：不定形式不会立即中断整个计算过程
- **保守而非猜测**：信息不足时返回较粗结果，不给出未经证明的精确答案
- **延迟 side condition**：非零、有限、正性和连续性等条件只在最终需要时出现
- **保持普通等式能力**：到达精确结果后仍可使用标准重写、代入和代数工具
- **规则可组合**：联合传递性和精度单调性允许规则在复杂表达式内部稳定传播
- **语义与计算分离**：传统定义保证数学正确性，表达式层提供组合性和自动化
- **跨任务复用**：极限、导数以及未来其他计算任务可以共享同一套协议

7. 关键可靠性约束

扩展新的计算系统时必须保证：

- 每一次信息退化都是语义安全的保守概括
- 精确结果确实具有刚性，不能由不同结果非平凡地退化得到
- 抽象运算可以不够精确，但不能给出错误的精确结果
- 运算的正向精度传播和逆向条件恢复必须分别验证
- $=?$ 只有在右侧精确时才能被当作实际计算结论使用
- 只有到达精确端点后，才能把多态相等式作为普通等式进行任意重写

8. 总结

该系统本质上是一个经过验证的多精度表达式计算框架

它允许中间计算保守地降低精度，用 $=?$ 延迟 side condition，并在最终结果足够精确时恢复普通等式和传统数学结论

其核心原则是：

中间计算允许不完全，但必须始终正确，最终精确结论必须严格，并能够回到普通数学推理